

Documento de Estrategia

Diseño lógico de ALU32 y ALU3 en Nand2Tetris HDL

Repositorio: dos implementaciones de ALU con sus archivos HDL, pruebas .tst y resultados esperados .cmp.

1. ALU32

1.1 Objetivo y arquitectura

ALU32 recibe x_1 , y_1 , x_2 y y_2 como buses de 16 bits. x_1/y_1 representan la mitad superior y x_2/y_2 la mitad inferior. La estrategia divide el problema en dos bloques de 16 bits y conserva el acarreo entre ambos durante la suma.

DIAGRAMA DE ARQUITECTURA

X: $x_1 \mid x_2$

Y: $y_1 \mid y_2$

$x_2, y_2 \rightarrow [\text{ALU inferior } 16] \text{ c15 } [\text{ALU superior } 16] \leftarrow x_1, y_1$

$[\text{ALU superior}] \mid [\text{ALU inferior}] \rightarrow \text{out1} \mid \text{out2}$

1.2 Preparación de entradas

$X \rightarrow \text{Mux16}(zx) \rightarrow \text{Not16} \rightarrow \text{Mux16}(nx) \rightarrow X \text{ preparado}$

$Y \rightarrow \text{Mux16}(zy) \rightarrow \text{Not16} \rightarrow \text{Mux16}(ny) \rightarrow Y \text{ preparado}$

1.3 Selección funcional

Se calculan $X \text{ AND } Y$ y $X + Y$ en paralelo. f selecciona una de las dos operaciones y no permite invertir el resultado final.

f	Operación
0	$X \text{ AND } Y$
1	$X + Y$

1.4 Suma y acarreo

$x_2 + y_2 \rightarrow \text{FullAdder16} \rightarrow \text{c15}$

$x_1 + y_1 + \text{c15} \rightarrow \text{FullAdder16} \rightarrow \text{overflow}$

La parte inferior debe calcularse primero para que el acarreo llegue correctamente a la parte superior.

1.5 Banderas

zr: se verifica que out1 y out2 sean completamente cero.

ng: $\text{ng} = \text{out1}[15]$, porque out1 contiene los bits 31..16.

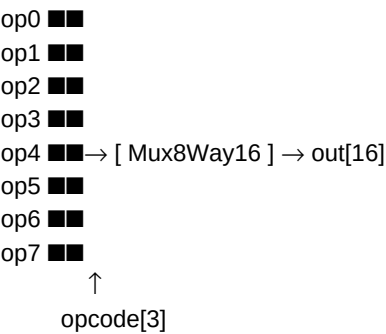
overflow: en esta implementación corresponde al carryFinal de la suma de la mitad superior.

2. ALU3

2.1 Objetivo

ALU3 recibe X, Y y Z de 16 bits y un opcode de 3 bits. Como $2^3 = 8$, existen ocho operaciones posibles. Todas se calculan y un Mux8Way16 selecciona la salida.

DIAGRAMA DE SELECCIÓN



2.2 Operaciones

Opcode	Operación
000	$X + Y + Z$
001	$(X \text{ AND } Y) \text{ OR } Z$
010	$(X \text{ OR } Y) \text{ AND } Z$
011	$X \text{ AND } Y \text{ AND } Z$
100	$X \text{ OR } Y \text{ OR } Z$
101	$X + Y - Z$
110	$(\text{NOT } X) \text{ AND } Y \text{ AND } Z$
111	$-(X + Y + Z)$

2.3 Subexpresiones compartidas

Se calculan una sola vez $X \text{ AND } Y$, $X \text{ OR } Y$ y $X + Y$. Estas señales se reutilizan en varias operaciones, evitando duplicación de lógica.

2.4 Resta y negación

La resta usa complemento a dos: $X + Y - Z = X + Y + (\text{NOT } Z + 1)$. La operación 111 usa el mismo principio: $-(X + Y + Z) = \text{NOT}(X + Y + Z) + 1$.

2.5 Banderas

zr: se revisan los bytes out[0..7] y out[8..15] con Or8Way y se invierte el resultado combinado.

ng: ng = out[15].

3. Respuesta analítica

3.1 ALU32

La división en dos bloques de 16 bits permite reutilizar componentes y simplificar el circuito. La condición fundamental es propagar el acarreo desde la mitad inferior hacia la superior. Así se conserva el comportamiento de una suma de 32 bits.

3.2 ALU3

El opcode de 3 bits produce ocho posibilidades. Construir cada operación y seleccionarla mediante Mux8Way16 hace explícita la relación entre código y operación. La reutilización de subexpresiones reduce la repetición de compuertas.

3.3 Complemento a dos

$$-X = \text{NOT } X + 1$$

$$X + Y - Z = X + Y + \text{NOT } Z + 1$$

$$-(X + Y + Z) = \text{NOT}(X + Y + Z) + 1$$

4. Verificación

Cada carpeta contiene .hdl, .tst y .cmp. Las pruebas establecen las entradas, seleccionan las operaciones, ejecutan eval y comparan los resultados generados con los valores esperados. ALU3 recorre los ocho opcodes; ALU32 verifica las dos mitades, la propagación del acarreo y las banderas.

5. Conclusión

ALU32 extiende el procesamiento de 16 a 32 bits mediante dos bloques coordinados. ALU3 utiliza tres operandos y ocho operaciones mediante resultados intermedios y Mux8Way16. Ambas soluciones mantienen una estructura modular y verificable en el Hardware Simulator.